

An opinionated delivery platform for enterprise software.

The pipeline era is over.

Oyzu generates build, scan, and publish steps from application requirements and organizational policy. Teams don't author or maintain the sequence.

APPLICATION TEAMS

Declare
intent.

ORGANIZATIONS

Define
policy.



OYZU

Delivers.

</> One governed path
from development to production.

🔧 Delivery evolves through policy,
not pipeline rewrites.

🛡️ Security and auditability
built in.

AI accelerates code creation. Enterprise delivery must keep pace.



More code to deliver

↑ 98% more merged pull requests

↑ 154% larger pull requests on average



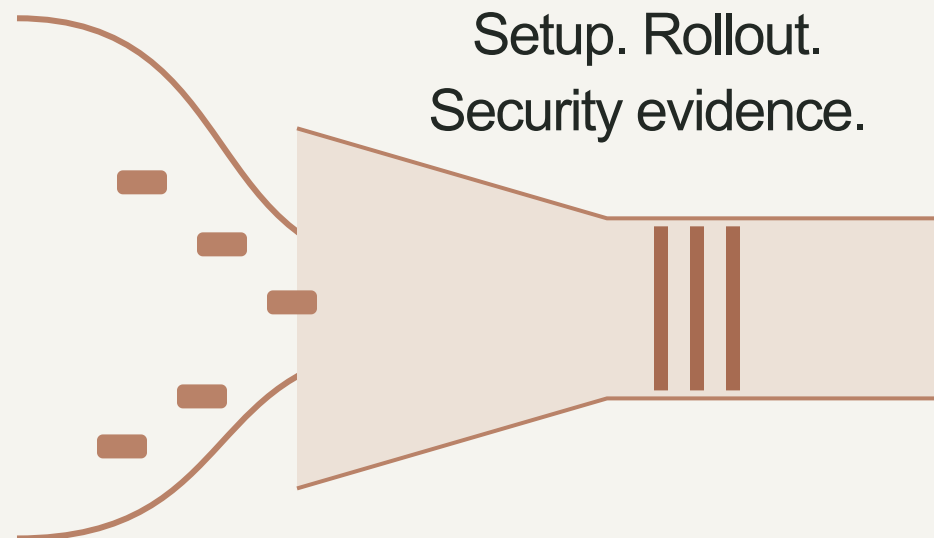
More change to absorb

New vulnerabilities

New controls and scanners

THE SAME DELIVERY BOTTLENECKS

Setup. Rollout.
Security evidence.



Delayed delivery

Changes wait to ship.



Engineering toil

Capacity goes to upkeep.



Slower security response

Protections take longer to reach every app.

The bottleneck slows both innovation and protection.

Developer friction consumes engineering capacity.

ENVIRONMENT READINESS

67%

of engineering leaders say their developers cannot build and test dev environments within 15 minutes.¹

Environment setup remains a hurdle.

ORGANIZATIONAL FRICTION

50%

of developers report that they lose 10 or more hours of their time every week because of organizational inefficiencies.²

Finding information. New technology. Switching between tools.



A working setup can still be an unapproved setup.

Developers and AI agents need approved tools, trusted sources, and company access rules. Public instructions don't establish company approval.

1. [Harness, State of Software Engineering Excellence 2025](#)

2. [Atlassian, State of Developer Experience 2025](#)

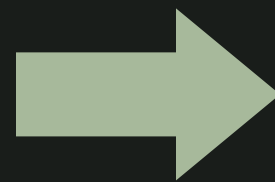
Every new standard becomes another rollout project.

New enterprise requirement



Adopt a new security scanner.

One organization-wide decision.



Barriers to adoption

TEAM A	Still on the old template CONFIGURATION WORK
TEAM B	Blocked by compatibility ADOPTION BLOCKED
TEAM C	Waiting for rollout ADOPTION PENDING
TEAM D	Operating under an exception COVERAGE UNRESOLVED

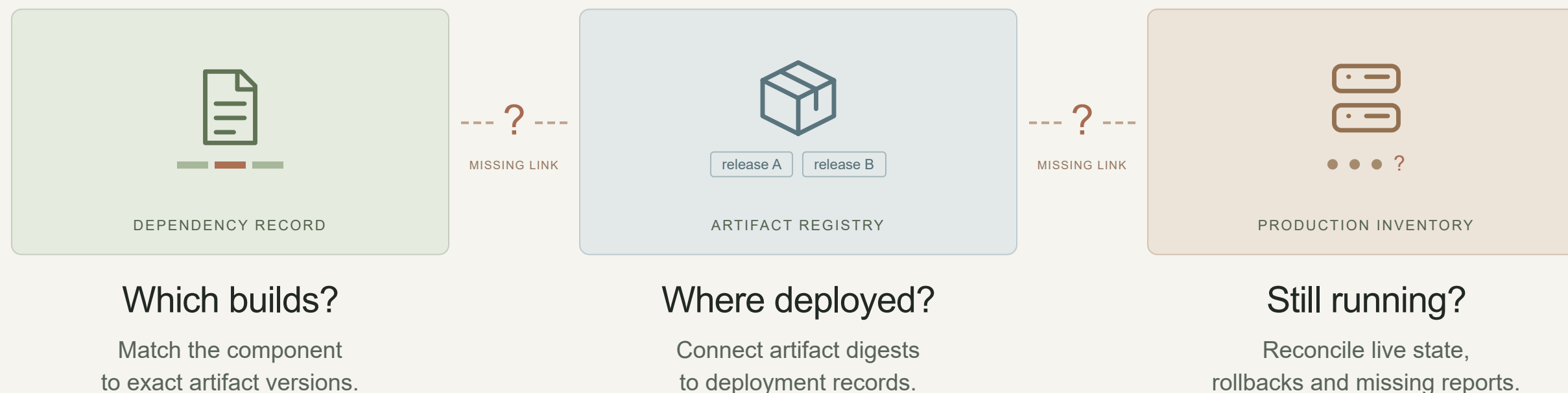
Engineering time diverted. Rollouts delayed. Control gaps persist.

The standard changes once. The adoption work repeats across teams.

A vulnerability alert becomes an investigation.

• Vulnerability detected

What is exposed **right now**?



Slower containment. More time proving what happened.

Disconnected records turn exposure assessment and audit proof into reconstruction work.

Three responsibilities. One system.

01

APPLICATION TEAMS

Declare intent.

Describe the application and outcome.
Keep native project files.

“Deliver this service to staging.”

02

ORGANIZATIONS

Define policy.

Set approved tools, checks, evidence
and delivery conditions.

“Only approved, verified artifacts.”

03

OYZU

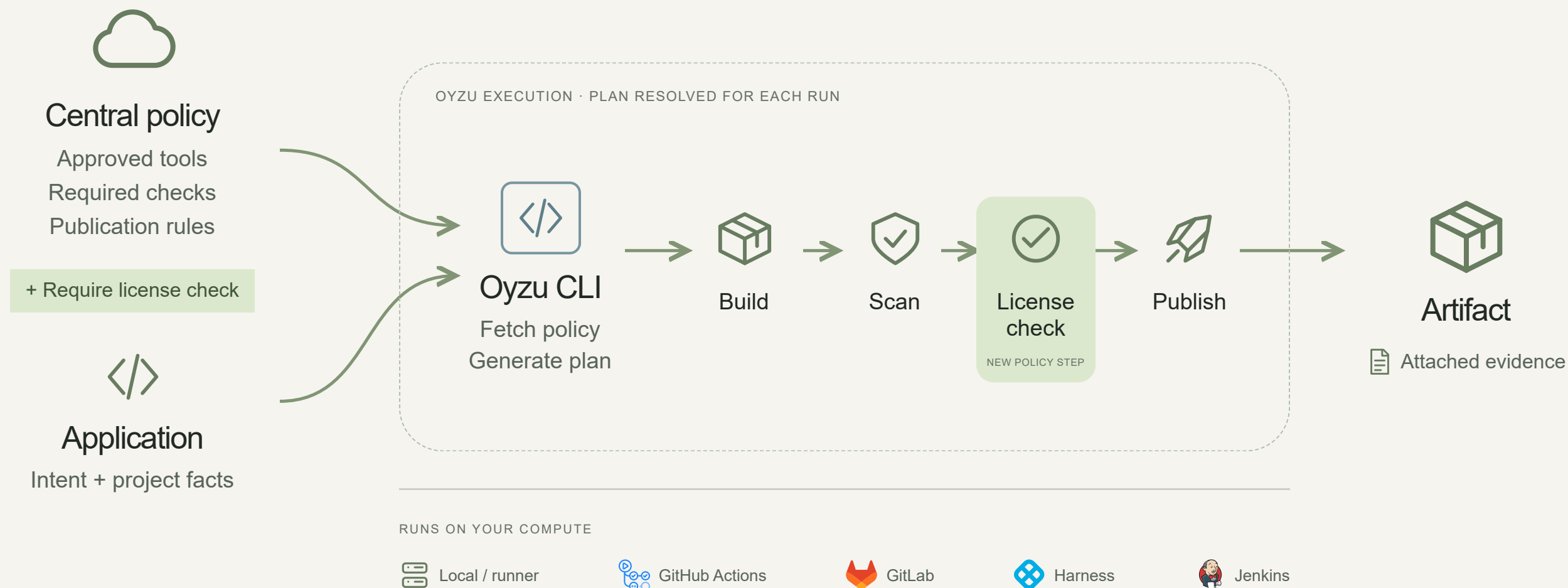
Deliver.

Generate the work. Execute the plan.
Retain proof of the outcome.

Source → artifact → production

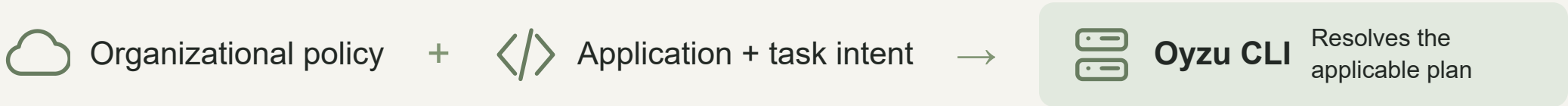
The steps still exist. Oyzu generates and maintains them.

Your CI provides compute. Oyzu determines the work.

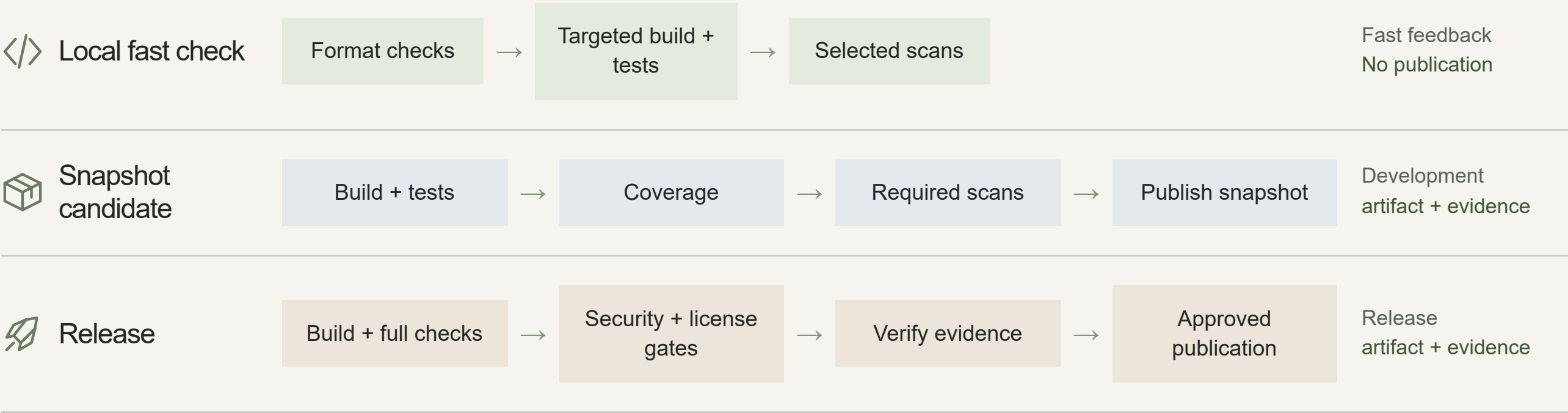


A new policy adds a check to the next applicable plan.
No application pipeline or vendor-specific template to rewrite.

Same CLI. Same policy system. The right checks for the task.



Example plans selected by task and policy

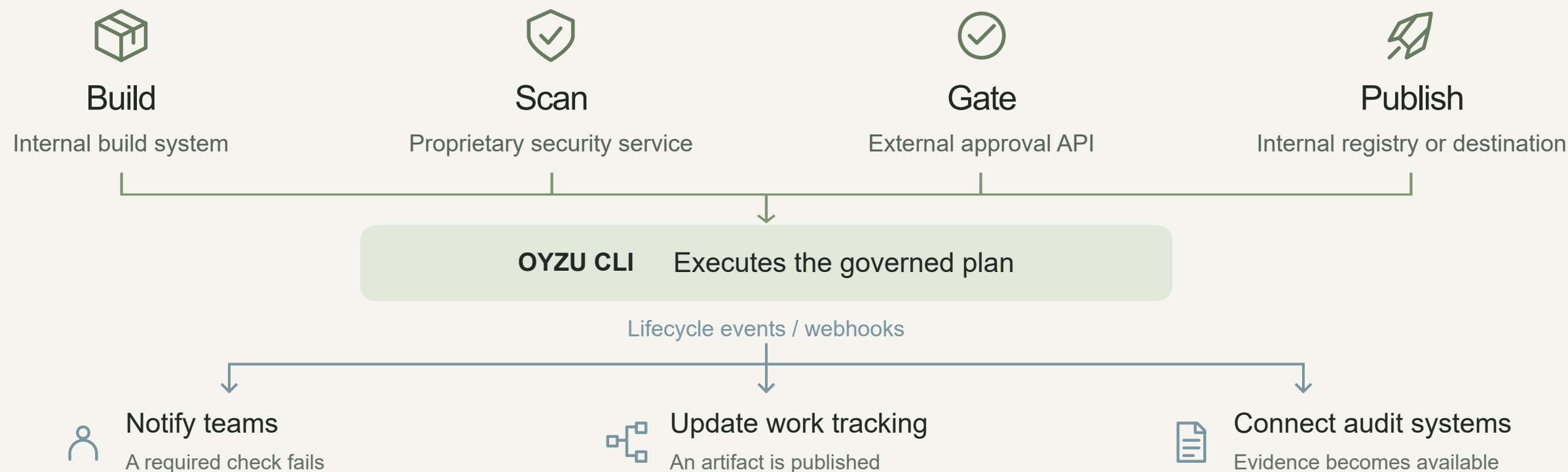


One governed toolchain. Short feedback loops locally. Required gates before publication.

Opinionated execution. Extensible integrations.

 Policy governs approved capabilities, execution gates and retained evidence.

PLUGINS PARTICIPATE IN THE WORK



Extend what runs. Connect what happens. Keep policy in control.

“Can’t AI just write the pipelines?”

THE SHORT ANSWER

Writing it is only the beginning of owning it.

AI can help create and maintain a pipeline. The customer still owns the workflow and its obligations.

Validate

Update

Govern

Integrate

Collect evidence

WHAT OYZU REMOVES

The application pipeline
your team has to own.

Teams declare intent. Organizations define policy.
Delivery expertise is built into the product.

No application pipelines to author. No pipeline templates to maintain.

One requirement. One governed rollout.

ONE SHARED REQUIREMENT Require a license assessment before publication.

01 · CAPABILITY

Ship support.

Release the integration through Oyzu. Identify compatible CLI versions.

02 · VALIDATION

Pilot it.

Test a rollout group. Surface compatibility issues and exceptions.

03 · POLICY

Activate it.

Set scope, approved versions, deadlines and expiring exceptions centrally.

04 · ENFORCEMENT

Apply it.

Compatible clients generate the required work. Older clients receive upgrade requirements.

POLICY-CONTROLLED ADOPTION

Notify → Restrict publication → Require upgrade

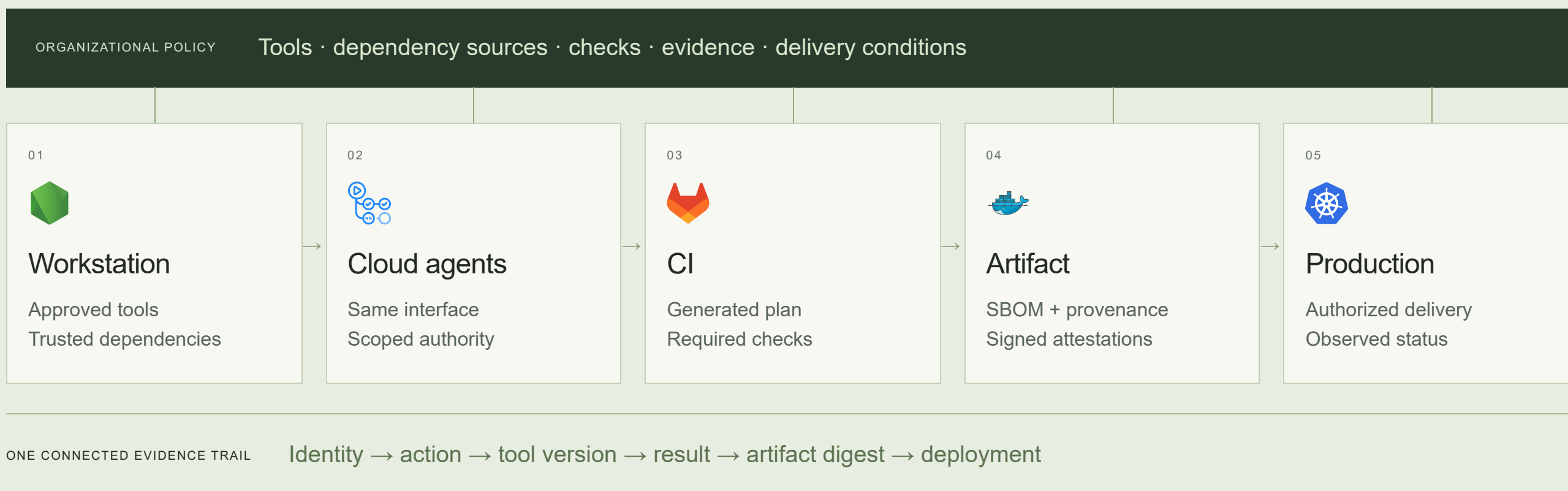
Platform authorization gates managed credentials and publication.

ONE MANAGED MECHANISM

Developer machines Cloud agents Existing CI

Roll out standards. Not repository migrations.

One policy. From first install to production.



The integration is the product.

Oyzu manages policy distribution, credential coordination, evidence retention and verification.
Customers keep their existing execution and delivery infrastructure.

The artifact is the record.

Every action. Every attestation. Every observed deployment.

payments-api / release B

Artifact detail

PRODUCT CONCEPT

ARTIFACT IDENTITY

One artifact.
Connected history.

SOURCE

Captured revision + tree

TOOLCHAIN

Runtime + builder identities

POLICY

Revision + resolved decisions

ARTIFACT

Content digest

SBOM

Provenance

Attestations

ACTIONS & OBSERVED STATUS

●

Build completed

Succeeded

Native commands, logs and test results retained

●

Release eligibility checked

Authorized

Required evidence verified against current policy

●

Artifact published

Published

Destination receipt bound to the artifact digest

●

Staging deployment observed

Deployed

Deployment event links this release to its target

●

Production deployment

Not confirmed

No deployment receipt has been reported

One record across connected build systems and deployment destinations.

Missing evidence stays visible. Success requires proof.

Shipping is only half the job. Then comes proving it.

REPORTED COMPLIANCE WORKLOAD

~10

hours / week

Policy reviews. Evidence collection.
Vendor attestations.

12 working weeks a year

Vanta's reported annual equivalent

Survey of business and IT leaders; not an engineering-only measure.

THE OYZU EXPERIENCE

Evidence is a delivery output.

01 Define the required controls

Versioned policy identifies required checks, approvals and evidence.

02 Capture proof as work happens

Bind actions, identities, results and attestations to the exact artifact.

03 Retrieve the record when asked

Trace source to deployment. Export control-linked evidence and expose gaps.

Less reconstruction at audit time. More traceability throughout delivery.

Who owns the delivery logic?

Powerful orchestration still leaves a workflow to own.
Oyzu makes generating that workflow the product’s responsibility.

	 GitLab	 Harness	 THE OYZU MODEL
Core approach	Integrated DevSecOps with CI/CD pipelines	Broad delivery orchestration with pipelines	Intent + native project facts + policy → execution plan
Reuse & governance	Auto DevOps, shared configuration and pipeline execution policies	Reusable pipeline / stage / step templates and policy as code	Policy governs generation; no application pipeline templates
Customer owns	Pipeline configuration or adopted defaults; customization and exceptions	Pipeline design or adopted templates; integrations and exceptions	Application intent, organizational rules and explicit exceptions
Where each is strong	Integrated source-to-delivery lifecycle	Deep delivery capabilities and orchestration flexibility	Governed tools from the workstation; product-owned delivery and connected evidence

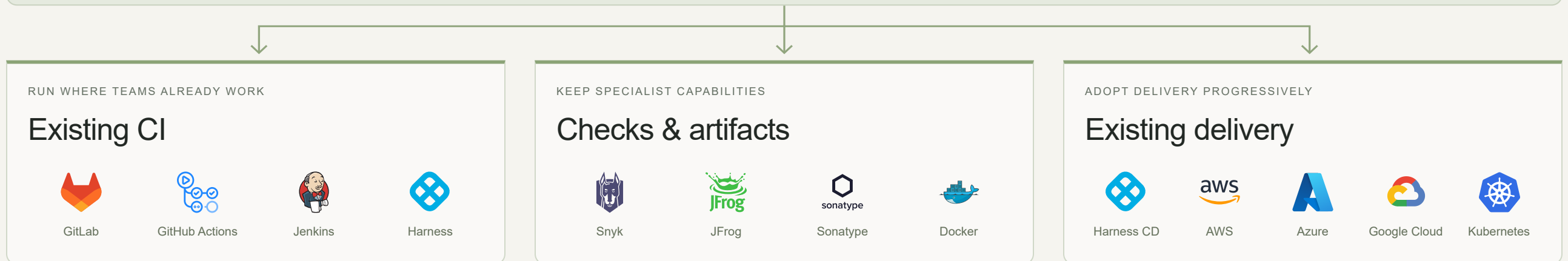
The bet: less customer-owned automation to build, roll out and maintain.

Keep the investment. Change the operating model.

INTEGRATION ROADMAP



Govern the first install. Generate delivery. Retain the proof.



Direct competition

Oyzu owns delivery logic and governance; existing platforms can provide execution.

Adopt alongside existing tools

Use existing execution and specialist delivery. Make Oyzu adoption possible without a wholesale migration.

The full delivery lifecycle.

A working build foundation.

WORKING TODAY

Native builds. Retained evidence.

The CLI plans and runs supported native builds, retaining artifacts, test reports and failure records.

[View build verification](#)

THIS ROUND

Governed publication.

Central policy, organizational identity, evidence services and publication gates.

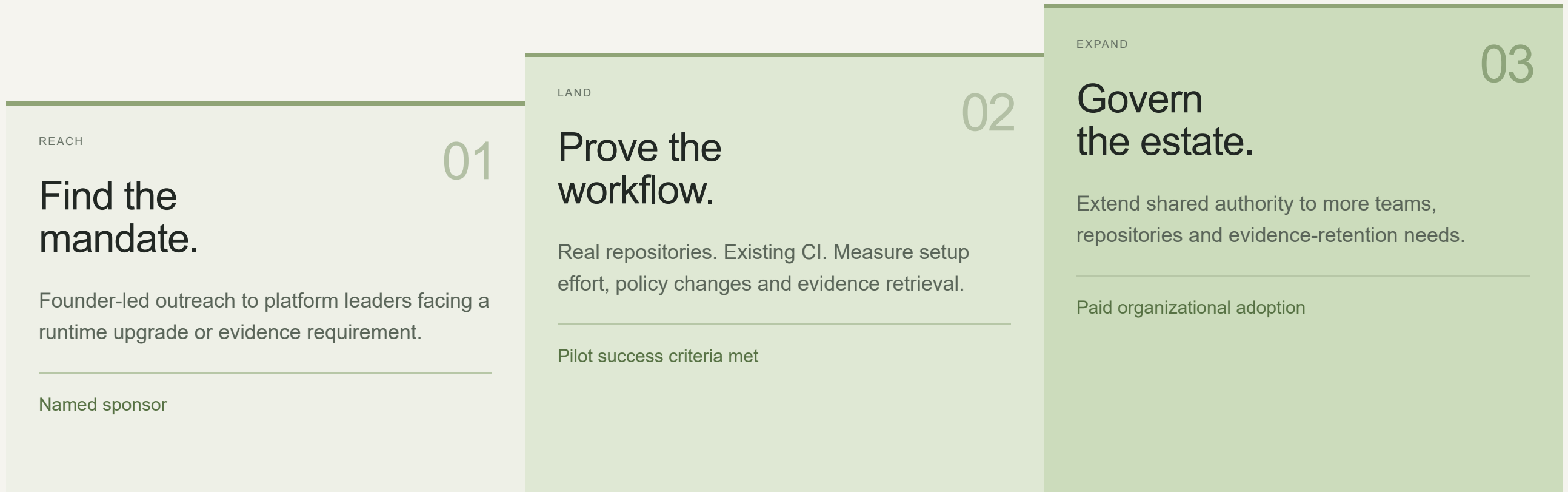
THEN

Broader delivery.

Extend the same governance and evidence model across more deployment destinations.

A working build foundation. A focused path to enterprise delivery.

One mandate. An organization-wide opportunity.



INITIAL BUYER Head of Platform Engineering / DevSecOps

Multiple application teams. Mixed CI. Recurring mandates.

Built for this problem.



FOUNDER

Micah Martin

Platform engineering leader.
Entrepreneur. Builder.

20

YEARS BUILDING
DEVELOPER INFRASTRUCTURE

BUILD

Engineering & DevOps leadership

Lifion by ADP · Engineering Manager, DevOps
SoulCycle · Director of Engineering

GOVERN

Enterprise operating experience

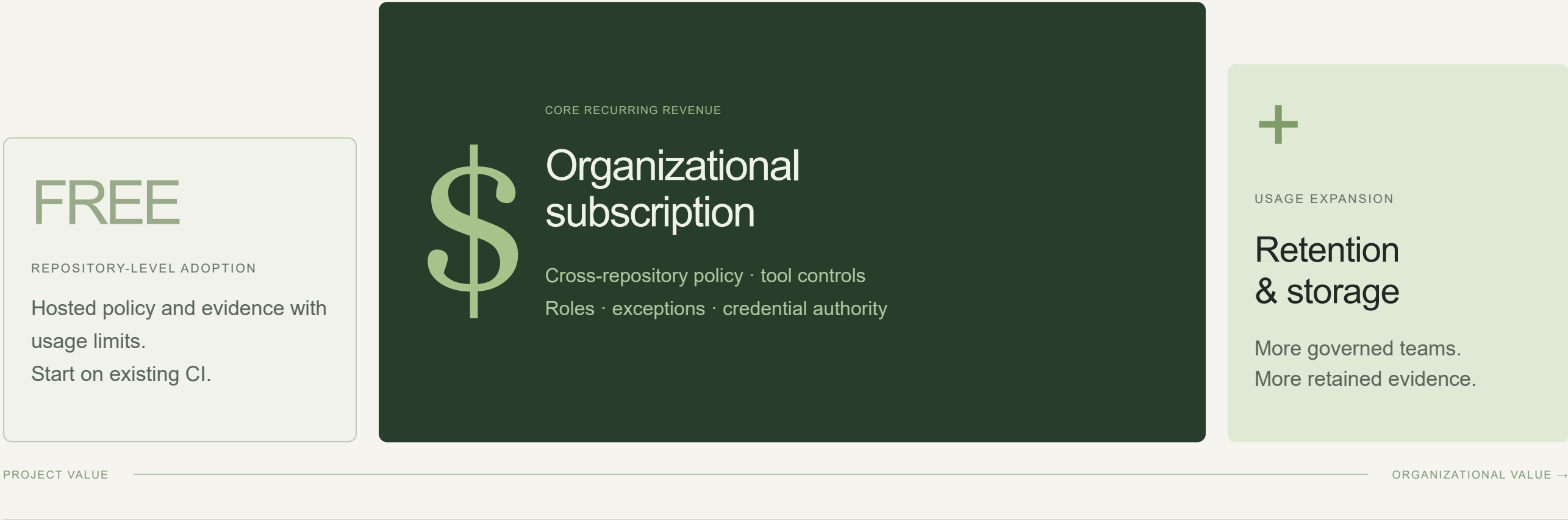
Current AVP, Platform Engineering · major enterprise

SELL

Customer-facing delivery expertise

Harness · Solutions Architecture → Director · 2020-2024

Adoption earns trust.
Governance earns revenue.



Existing execution first. Optional Oyzu compute becomes a separate usage business later.

SUBSCRIPTION + USAGE

\$1M

To make governed delivery
a repeatable enterprise product.



- \$600K
Product & engineering
- \$250K
Customer validation & onboarding
- \$150K
Operations & reserve

3 Design partners

6 Independent repositories
across ≥ 2 CI systems

2 Paying organizations

ROUND EXIT TARGETS · ONBOARDING ≤ 1 ENGINEER-DAY PER SUPPORTED PROJECT

Appendix

Product evidence. Economics. Operating plan. References.

Any stack. Same command.

LOCAL TERMINAL · IN YOUR PROJECT

01 · PREVIEW

```
$ oyzu build --plan
```

Inspect the plan before executing build actions.

02 · BUILD

```
$ oyzu build
```

Run supported native build actions.

03 · SCAN

```
$ oyzu scan
```

Run applicable security checks and retain findings.

04 · PUBLISH WHEN READY

```
$ oyzu publish
```

Infer the build output, verify it, and authorize publication.

Every project starts with oyzu build.

Oyzu detects the stack and runs the right native tools.
Developers use the same command across projects.



Successful build

Artifact and reports retained.

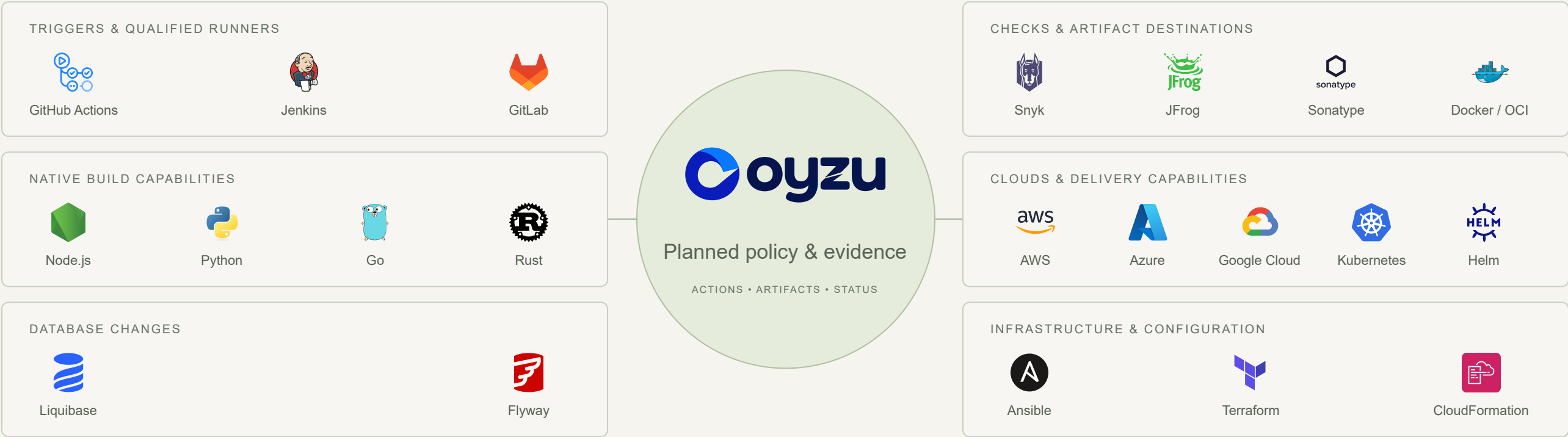


Failed test or coverage gate

Failure evidence retained for diagnosis.

Local builds give feedback. Publication applies the
required authorization and evidence checks.

The connected ecosystem.



Know the artifact

Source · digest · SBOM

Verify the history

Actions · attestations · provenance

Explain the outcome

Policy decisions · status · deployments

Clouds, database changes and infrastructure belong in the same governed lifecycle.

Inside the native build evidence.

 GitHub Actions

8 / 8 JOBS SUCCEEDED

PUBLIC RUN · 36863353630

CLI build / checks	macOS ✓ Linux ✓ Windows ✓
Compiled CLI task scenarios	macOS ✓ Linux ✓ Windows ✓
Captured-source builds	Linux ✓
Isolated BuildKit worker contract	Linux ✓

[Inspect the actual workflow run ↗](#)

CURRENT IMPLEMENTATION

Source → plan → native build → bundle

Initial ecosystem profiles, offline container actions and digest-verifiable outputs.

NEXT PRODUCT MILESTONE

Complete governed publication

Managed policy, evidence lifecycle, identity and tested publication integrations.

Next milestone: publish artifacts through centrally governed policy and evidence services.

What repeated delivery work can cost.

ILLUSTRATIVE LABOR COST PER CHANGE



$\text{Repositories} \times 4 \text{ hours} \times \$100 / \text{loaded hour}$

WORK OYZU AIMS TO REDUCE

Repeated workflow edits.
Toolchain coordination.
Evidence reconciliation.

Net value includes onboarding, subscription and ongoing operating costs.

Example assumptions: four hours per repository at \$100 per hour.

How the \$1M is allocated.

ILLUSTRATIVE 12-MONTH COSTS · USD

Founder / product engineering	\$120K
Two engineers · \$180K loaded each	\$360K
Security / integration specialists	\$120K
Product & engineering	\$600K
Solutions / onboarding capacity	\$150K
Partner evaluation + acquisition · \$50K each	\$100K
Cloud / evidence services	\$60K
Legal, finance & administration	\$40K
Reserve	\$50K
Total planned allocation	\$1M

12-MONTH TARGETS

Qualification

Supported publication succeeds with required evidence; missing evidence blocks completion and retries preserve receipts.

Onboarding

Six independent repos across ≥ 2 CI systems; ≤ 8 engineer-hours per supported repo after prerequisites are ready.

Measured value

Compare policy-rollout labor and evidence-retrieval time against partner baselines; include exceptions and operating costs.

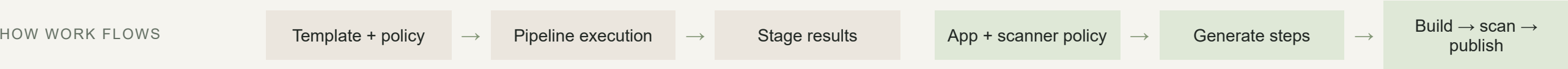
Commercial conversion

Convert two successful pilots into paid organizational subscriptions.

Oyzu vs. Harness

A new security scanner becomes mandatory.

	 Harness	Oyzu
Starting strength	Shared templates centralize logic. Pinned versions and input changes can still require upgrades across consuming pipelines.	Generates build, scan and publish steps from application requirements and policy. Teams do not maintain that sequence.
Teams configure	Pipeline templates, inputs and OPA guardrails.	Application requirements; security selects the scanner, scope and blocking rules.
A requirement changes	Stable references can follow updates automatically; pinned versions and changed inputs need adoption or reconciliation.	Security updates policy. Oyzu includes the required scanner in the next applicable generated plan.
Logic is maintained by	Platform teams maintain pipeline steps and integrations.	Oyzu maintains scanner integration code centrally. The organization owns rules and exceptions.



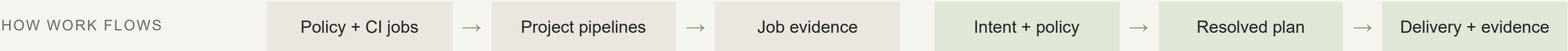
| The steps still exist. Oyzu generates the sequence and maintains the integrations.

[Harness templates](#) · [Harness policy as code](#) · [Version adoption and reconciliation](#)

Oyzu vs. GitLab

Require a security check across many projects.

	 GitLab	Oyzu
Starting strength	Pipeline execution policies enforce shared CI/CD jobs across projects, including projects without their own CI configuration.	One governed path connecting approved tools, delivery controls and retained evidence.
Teams configure	Shared CI jobs and pipeline execution policies.	Application intent, approved tools and organizational policy.
A requirement changes	Enforce shared jobs centrally across scoped projects.	Change shared policy; Oyzu resolves the supported plan and records enforcement.
Logic is maintained by	Platform teams maintain CI definitions and application compatibility.	Oyzu maintains supported delivery mechanics; the organization owns policy and exceptions.



Oyzu brings the governed model to the developer command and the delivery plan.

[GitLab pipeline execution policies](#)

Oyzu vs. Bazel

Approve a new compiler across independent repositories.

	Bazel	Oyzu
Starting strength	Build rules and platform-aware toolchain resolution select tools for reproducible, explicitly modeled builds.	One governed path connecting approved tools, delivery controls and retained evidence.
Teams configure	Build targets, rules and registered toolchains.	Application intent, approved tools and organizational policy.
A requirement changes	Publish approved rule/toolchain versions; adopt them across workspaces.	Change shared policy; Oyzu resolves the supported plan and records enforcement.
Logic is maintained by	Rule owners maintain build definitions and toolchain integrations.	Oyzu maintains supported delivery mechanics; the organization owns policy and exceptions.
HOW WORK FLOWS	Rules + toolchains → Build graph → Build outputs	Intent + policy → Resolved plan → Delivery + evidence

Tool selection becomes part of an organization-wide delivery model.

Oyzu vs. Pants

Standardize approved tools for developers and AI agents.

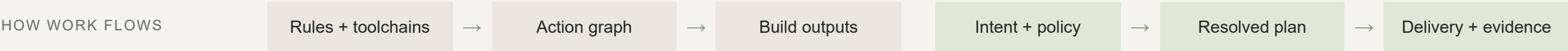
	Pants	Oyzu
Starting strength	Dependency inference and isolated execution reduce build configuration and make supported tasks repeatable.	One governed path connecting approved tools, delivery controls and retained evidence.
Teams configure	Repository settings, backends and tool versions.	Application intent, approved tools and organizational policy.
A requirement changes	Update shared configuration or plugins; coordinate repository adoption.	Change shared policy; Oyzu resolves the supported plan and records enforcement.
Logic is maintained by	Teams maintain configuration and extensions around inferred tasks.	Oyzu maintains supported delivery mechanics; the organization owns policy and exceptions.
HOW WORK FLOWS	Config + inference → Isolated tasks → Task outputs	Intent + policy → Resolved plan → Delivery + evidence

Convenient setup and organizational approval belong in the same experience.

Oyzu vs. Buck2

Move teams onto an approved production toolchain.

	Buck2	Oyzu
Starting strength	Reusable rules separate build behavior from tool sourcing. Hermetic toolchains can download and track tools explicitly.	One governed path connecting approved tools, delivery controls and retained evidence.
Teams configure	Build rules and production toolchain definitions.	Application intent, approved tools and organizational policy.
A requirement changes	Distribute toolchain changes and coordinate repository adoption.	Change shared policy; Oyzu resolves the supported plan and records enforcement.
Logic is maintained by	Rule owners maintain tool sourcing and production toolchains.	Oyzu maintains supported delivery mechanics; the organization owns policy and exceptions.



| The product takes on the recurring work around the build engine.

More software. More downstream pressure.

GITHUB · 2025

+25.1%

Code activity

Year-over-year growth in code pushes / commits across GitHub.

FAROS · 2026

+71.8%

Average PR size

Larger changes in the study's higher-AI-adoption periods.

FAROS · 2026

+125.4%

Monthly incidents

More incident volume in the same dataset.

Separate studies, consistent pressure on downstream systems.

GitHub measures platform-wide activity. Faros covers 22,000 developers across 4,000 teams; its 2026 sample differs from the 2025 study.

Faros 2025 also observed 91% longer review time with higher AI adoption, alongside 98% more merged pull requests and 154% larger average pull requests. This measures code-review pressure, not post-merge delivery time.

Price the work. Measure the reduction.

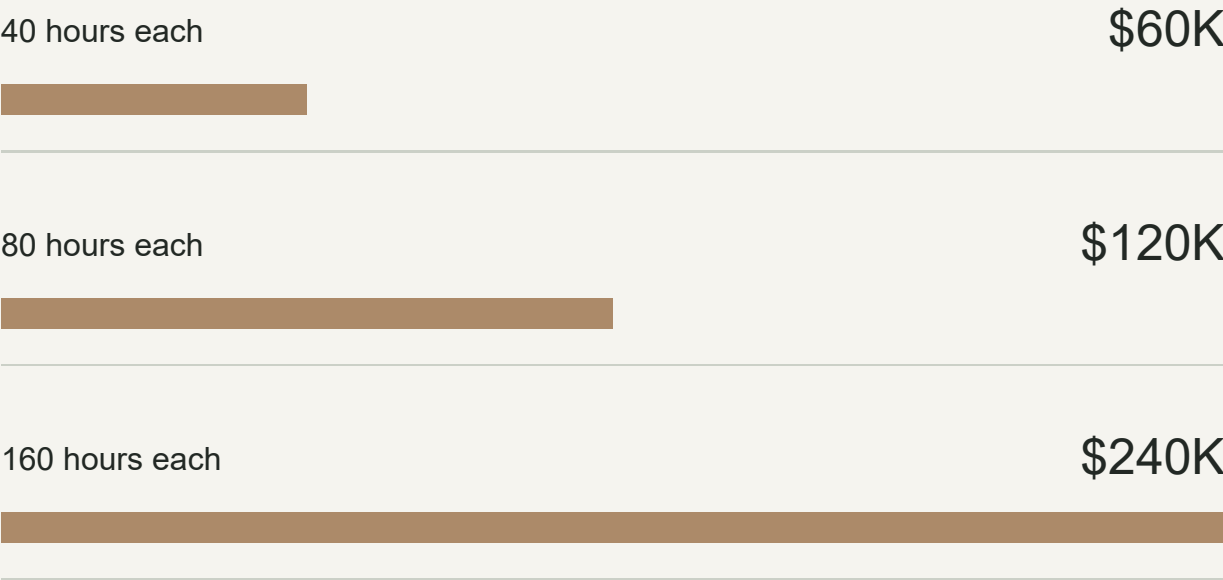
ILLUSTRATIVE ANNUAL EVIDENCE-PREPARATION LABOR

\$120,000

10 contributors × 80 hours per audit cycle
× \$150 loaded hourly cost × 1 cycle per year

Example assumptions for a single annual audit cycle.

SENSITIVITY · SAME 10 PEOPLE AND \$150 / HOUR



Pilot measurement

Engineering, platform and security hours spent locating records, reconciling gaps and answering requests. Compare the same audit scope before and after.

Additional cost drivers

Remediation and reassessment can add expense; delayed customer approvals can affect revenue.

Complement the GRC stack

Oyzu supplies technical delivery evidence. GRC tools and auditors assess broader controls, exceptions and effectiveness.

References.

Industry & prior art

Harness Engineering Excellence 2025 · Atlassian Developer Experience 2025
Environment readiness and organizational friction.

Faros AI Productivity Paradox, 2025 · Speed Trap, 2026
Observed output and downstream workload.

GitHub Octoverse 2025
Platform-wide code activity growth.

GitLab Auto DevOps
Detection, build/test/security and deployment automation.

Dagger · Cloud Native Buildpacks
Portable code-based pipelines; source-to-image conventions.

GitHub reusable workflows
Established workflow sharing and reuse.

Product & research evidence

Vanta State of Trust 2025
Reported compliance workload.

Oyzu workflow run 36863353630
Eight successful jobs; historical, scoped verification.

Oyzu implementation status
Current profiles, measured capabilities and gaps.

Micah Martin's career history
Platform engineering, enterprise governance and customer-facing delivery.